

CPU AND REGISTER ORGANIZATION

The CPU of the PIC16F72 consists of the main processing unit along with registers used for instruction execution, data manipulation, memory addressing, and control.

The important registers and components related to CPU operation include:

- Working (W) register
- STATUS register
- Program Counter (PC)
- PCL and PCLATH registers
- File Select Register (FSR)
- Special Function Registers (SFRs)
- General Purpose Registers (GPRs)

Working Register (W)

The W register is an 8-bit working register used by the CPU for temporary storage of data during instruction execution. Many arithmetic, logical, and data-transfer instructions use the W register.

For example, the instruction:

`MOVLW 25H`

loads the value 25H into the W register.

The W register is therefore an important part of the instruction execution process and needs to be represented in the simulator.

File Registers

The PIC16F72 uses file registers for storing and accessing data. They can be broadly divided into:

Special Function Registers (SFRs): Used to control and monitor different functions of the microcontroller.

General Purpose Registers (GPRs): Used by the program for storing temporary data.

Some important registers include the STATUS register, PCL, PCLATH, FSR, and I/O port registers.

STATUS Register

The STATUS register contains information about the result of arithmetic and logical operations. It also contains bits used for register-bank selection and other processor status information.

The main status bits relevant to CPU operation are:

- C – Carry flag
- DC – Digit Carry flag
- Z – Zero flag
- TO – Time-out status bit
- PD – Power-down status bit
- RP0 and RP1 – Register bank selection bits

PROGRAM COUNTER

The Program Counter (PC) is a register that contains the address of the instruction to be executed. The PIC16F72 has a 13-bit Program Counter.

During normal sequential execution, the Program Counter advances to the address of the next instruction.

For example:

Instruction Address	Instruction
0000	MOVLW 10H
0001	MOVWF 20H
0002	ADDWF 21H
0003	...

After executing an instruction, the PC normally moves to the next instruction address.

However, instructions that change program flow, such as GOTO, CALL, and RETURN, can modify the normal sequence of execution.

PCL and PCLATH

The Program Counter is associated with two registers:

PCL (Program Counter Low): Contains the lower portion of the Program Counter.

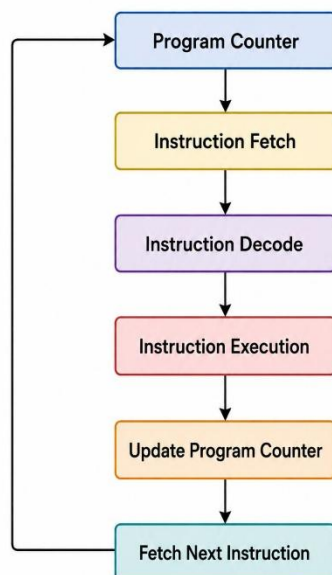
PCLATH (Program Counter Latch): Provides the higher bits required for certain Program Counter operations.

These registers are important when implementing program-control instructions in the simulator.

Program Counter Operation in the Simulator

The simulator should maintain the current Program Counter and use it to determine which instruction is fetched next.

The basic instruction execution process can be represented as:



Stack Organization

The PIC16F72 uses a hardware stack for storing return addresses during subroutine calls and interrupts.

Unlike processors that have a conventional software-accessible Stack Pointer (SP) register, the PIC16F72 uses a dedicated hardware stack. Therefore, the stack should not be treated as a normal data-memory stack with a directly accessible SP register.

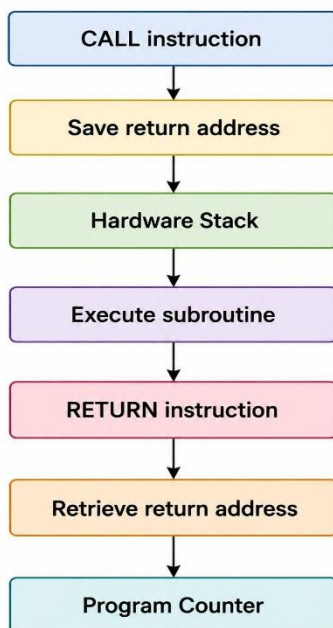
The hardware stack is used mainly by instructions such as:

- CALL
- RETURN

When a CALL instruction is executed, the return address is stored in the stack. The processor then jumps to the subroutine.

When the RETURN instruction is executed, the stored return address is retrieved and loaded into the Program Counter.

The basic operation is:



The PIC16F72 provides an 8-level hardware stack.

Stack Implementation in the Simulator

The simulator should maintain an internal representation of the hardware stack. It can be implemented using an array or another suitable data structure.

The simulator should support:

Pushing a return address when a CALL occurs.

Popping the return address when a RETURN occurs.

Maintaining the correct stack level.

Handling stack operations associated with interrupts.

The internal stack representation is an implementation feature of the simulator and should not be confused with an actual user-accessible Stack Pointer register in the PIC16F72.

STATUS and Flag Information

The STATUS register provides information about the current state of the CPU and the results of arithmetic and logical operations.

The important bits are described below.

PIC MICROCONTROLLER STATUS REGISTER (STATUS) BITS

BIT NAME	SYMBOL	DESCRIPTION
(C)	C	Indicates a carry out of the most significant bit (MSB) after an arithmetic operation (like addition or subtraction).
(DC)	DC	Indicates a carry out of the low-order nibble (Bit 3 to Bit 4) in an 8-bit arithmetic operation.
(Z)	Z	Indicates that the result of an arithmetic, logical, or data movement operation was zero.
(TO)	TO	Time-out: Indicates that a watchdog timer time-out condition occurred, leading to a reset or wake-up.
(PD)	PD	Power-down: Indicates the processor power status, where 1=Power-up/Reset and 0=Sleep mode.
(RP0)	RP0	Register Bank Select Bit 0: Combined with RP1, defines the current register bank (0-3) being accessed.
(RP1)	RP1	Register Bank Select Bit 1: Combined with RP0, defines the current register bank (0-3) being accessed.

Carry Flag

The Carry flag indicates whether an arithmetic operation produces a carry beyond the most significant bit of the 8-bit result.

For example:

```
11111111
+ 00000001
-----
1 00000000
```

The 8-bit result is 00000000, while a carry is generated. Therefore, the Carry flag is set.

Digit Carry Flag (DC)

The Digit Carry flag indicates whether a carry occurs from bit 3 to bit 4 during an arithmetic operation.

This flag is useful for detecting carry between the lower and upper nibbles of an 8-bit value.

Zero Flag (Z)

The Zero flag indicates whether the result of an operation is zero.

For example:

05H – 05H = 00H

Since the result is zero, the Zero flag is set.

If the result is non-zero, the Zero flag is cleared.

Register Bank Selection Bits

The RP0 and RP1 bits are used for selecting the appropriate register bank in the data-memory organization.

The simulator must consider these bits when determining which file register is being accessed.